Technical Specification

Local-First Financial Analysis Application — V1

Status: Draft v1 **Companions:** `docs/prd.md`, `docs/architecture.md` **Audience:** Implementing engineers / agents

---

1. Scope

This document decomposes the V1 architecture into the smallest modules that can be implemented independently in parallel. Each module has:

- a stable public interface (trait, struct, or component contract);
- explicit dependencies (which other modules' interfaces it uses);
- a definition of done (DoD) — what must compile, what must test green, what artifacts must exist;
- a single owner during implementation.

The decomposition principle: **a module is the smallest unit whose internal implementation can change without affecting any other module's public contract.**

2. Implementation slice

The architecture's full surface is large. This spec identifies a **V1-implementation slice** — the modules that must work end-to-end before any module is "done." Modules outside the slice are still specified here but are deferred to a follow-up implementation pass:

In V1 slice: M01–M37, M40, M43, M44, M45 (core data path: ingest one company via SEC `companyfacts`, normalize, store, render dashboard with summary widgets, charts, statements table, lineage panel, plus tests).

Deferred to follow-up: XBRL-XML fallback parser, 8-K Item 4.02 HTML parser, bundled ECB FX dataset, amendment-coverage-gap UI surfacing, FYE-change banner, restatement banner resolution UI, full Diagnostics tab. The architecture and this spec describe their interfaces so the slice does not block them.

3. Repository layout

```
econ_project/
├── docs/                    # PRD, architecture, this spec
├── src-tauri/               # Rust core (M01)
│   ├── Cargo.toml
│   ├── tauri.conf.json
│   ├── capabilities/
│   └── src/
│       ├── main.rs          # M01
│       ├── domain/          # M03 types
│       ├── errors.rs        # M04
│       ├── db/              # M02, M05
│       ├── repos/           # M06–M12
│       ├── sources/         # M13–M17
│       ├── normalize/       # M18–M21
│       ├── pipeline/        # M22–M27
│       ├── derived/         # M28
│       └── ipc/             # M29–M30
├── src/                     # React frontend (M31)
│   ├── api/                 # M32 typed IPC client
│   ├── state/               # M33 TanStack Query hooks
│   ├── styles/              # M34 Tailwind
│   ├── routes/
│   ├── components/
│   └── features/
│       ├── home/            # M35
│       ├── dashboard/       # M36–M39
│       ├── lineage/         # M40
│       └── diagnostics/     # M41
└── tests/
    ├── fixtures/            # M44
    └── e2e/                 # M45
```

4. Cross-cutting types

These types are owned by **M03 (domain)** and used by every other module. The contract is canonical; no other module redefines these.

```
// All currency / per-share values: INTEGER micro-units (×1,000,000)
pub type Micro = i64;
```

```rust
#[derive(Clone, Debug, PartialEq, Eq)]
pub struct Cik(pub String);            // 10-digit zero-padded

pub struct Ticker(pub String);

pub struct AccessionNo(pub String);    // e.g., "0000320193-24-000123"

pub enum FormType {
    TenK, TenQ, TenKA, TenQA, EightK, Other(String),
}

pub struct Filing {
    pub accession_no: AccessionNo,
    pub cik: Cik,
    pub form_type: FormType,
    pub filed_at: chrono::NaiveDate,
    pub period_of_report: Option<chrono::NaiveDate>,
    pub is_amendment: bool,
    pub amends: Option<AccessionNo>,
    pub item_4_02_8k: bool,
}

pub enum PeriodKind { Annual, Quarterly }

pub struct Period {
    pub id: i64,
    pub cik: Cik,
    pub fiscal_year: i32,
    pub fiscal_quarter: u8,            // 0 = annual, 1..=4 = quarterly
    pub fiscal_year_end: String,       // "MMDD"
    pub start_date: chrono::NaiveDate,
    pub end_date: chrono::NaiveDate,
    pub kind: PeriodKind,
    pub is_53_week: bool,
}

pub struct RawFact {
    pub id: i64,
    pub cik: Cik,
    pub accession_no: AccessionNo,
    pub taxonomy: String,              // 'us-gaap', 'dei', ...
    pub concept: String,
```

```
    pub unit: String,                  // 'USD', 'shares', 'USD/shares'
    pub value_numeric: Micro,          // §6.2 micro-unit convention
    pub period_start: Option<chrono::NaiveDate>,
    pub period_end: chrono::NaiveDate,
    pub is_instant: bool,
    pub fy: Option<i32>,
    pub fp: Option<String>,
    pub filed: Option<chrono::NaiveDate>,
    pub source_kind: SourceKind,
    pub ingested_at: chrono::DateTime<chrono::Utc>,
}

pub enum SourceKind { XbrlApi, XbrlXml }

#[derive(Clone, Copy, Debug, PartialEq, Eq, Hash)]
pub enum Metric {
    Revenue, CostOfRevenue, GrossProfit,
    OperatingIncome, NetIncome,
    EpsBasic, EpsDiluted,
    SharesOutstandingBasic, SharesOutstandingDiluted,
    CashAndEquivalents, LongTermDebt, CurrentDebt, TotalDebt,
    TotalAssets, TotalLiabilities, TotalEquity,
    CashFromOperations, CapitalExpenditures, DepreciationAmortization,
    HistoricalMarketCap, CurrentMarketCap,
}

pub struct NormalizedFact {
    pub id: i64,
    pub cik: Cik,
    pub metric: Metric,
    pub period_id: i64,
    pub value: Micro,
    pub unit: String,
    pub source_fact_id: i64,
    pub source_kind: SourceKind,
    pub is_primary: bool,
    pub original_value: Option<Micro>,
    pub original_unit: Option<String>,
    pub fx_rate_micro: Option<i64>,
    pub fx_rate_source: Option<String>,
    pub fx_rate_date: Option<chrono::NaiveDate>,
    pub superseded_by: Option<i64>,
```

```
}

pub struct DerivedMetric {
    pub id: i64,
    pub cik: Cik,
    pub formula_id: String,
    pub period_id: i64,
    pub value: Option<Micro>,
    pub is_complete: bool,
}

pub enum Severity { Info, Warn, Error }

pub struct IngestionEvent {
    pub id: i64,
    pub cik: Option<Cik>,
    pub accession_no: Option<AccessionNo>,
    pub stage: String,
    pub level: Severity,
    pub user_visible: bool,
    pub message: String,
    pub detail_json: Option<String>,
}

pub struct LineageRecord {
    pub fact: RawFact,
    pub filing: Filing,
    pub fx_conversion: Option<FxConversion>,
    pub supersedes: Vec<NormalizedFact>,  // backward chain walk
}

pub struct FxConversion {
    pub original_value: Micro,
    pub original_unit: String,
    pub rate_micro: i64,
    pub rate_source: String,
    pub rate_date: chrono::NaiveDate,
}
```

5. Module catalog

Each module's spec format:

> **MNN — name** *(layer)* **Owner agent role.** Brief purpose. **Depends on:** M..., M... **Public interface:** (trait / struct / file paths) **DoD:** what must build/test green.

Foundation (M01–M04)

M01 — Project skeleton

Architect. Tauri 2 application with Cargo workspace, frontend toolchain, build configuration, capability manifests. **Depends on:** none. **Public interface:** `src-tauri/Cargo.toml`, `src-tauri/tauri.conf.json`, `src-tauri/capabilities/default.json`, `package.json`, `vite.config.ts`, `tsconfig.json`. Tauri command registry stub. **DoD:** `cargo check` passes; `pnpm install` succeeds; `pnpm tauri dev` launches an empty window with "EconProject" title.

M02 — SQLite schema + migration runner

Developer. `0001_initial.sql` with the §6.3 schema; `refinery` integration; `db.rs` opens the DB with the §12.1 PRAGMAs. **Depends on:** M01. **Public interface:** `db::open(path) -> Pool` returning a connection pool with one writer + N readers; runs migrations on open. **DoD:** Test that opens an in-memory DB, runs migrations, asserts every table exists with the documented columns and indexes; integrity_check returns "ok".

M03 — Domain types

Architect. All §4 types in `src-tauri/src/domain/`; serde derives for IPC; `chrono` types for dates. **Depends on:** M01. **Public interface:** `domain::*` re-exports. **DoD:** Round-trip serde tests for every public struct/enum.

M04 — Error types

Developer. `errors.rs` — `SourceError`, `PipelineError`, `RepoError`, `AppError` (top-level for IPC) using `thiserror`. `AppError` has stable `code: &'static str`. **Depends on:** M01. **Public interface:** `errors::*`. **DoD:** Each error variant has unit tests for `Display` output and code mapping.

Persistence (M05–M12)

Convention: every repo struct takes a `Pool`-shaped reference and exposes async methods. Single-writer discipline is enforced at the pool level (M05). All financial-value reads return `Micro`.

M05 — Database connection management

Developer. WAL setup, single writer + read pool of 4, async accessors, integrity check on open. **Depends on:** M02. **Public interface:**

```
pub struct Pool { /* ... */ }
impl Pool {
    pub async fn write(&self) -> WriteGuard<'_>;
    pub async fn read(&self) -> ReadGuard<'_>;
    pub async fn integrity_check(&self) -> Result<(), RepoError>;
}
```

DoD: Concurrent test (4 readers + 1 writer) completes without `SQLITE_BUSY`; integrity_check passes after migrations.

M06 — Company repository

Developer. CRUD for the `company` table + ticker→CIK lookup join with cached `company_tickers`. **Depends on:** M03, M05, M14. **Public interface:**

```
#[async_trait]
pub trait CompanyRepo {
    async fn upsert(&self, c: &Company) -> Result<(), RepoError>;
    async fn get_by_cik(&self, cik: &Cik) -> Result<Option<Company>, RepoError>;
    async fn get_by_ticker(&self, t: &Ticker) -> Result<Option<Company>, RepoError>;
    async fn list_saved(&self) -> Result<Vec<Company>, RepoError>;
    async fn remove(&self, cik: &Cik, drop_cache: bool) -> Result<(), RepoError>;
}
```

DoD: In-memory unit tests covering each method; FK behavior (RESTRICT on remove unless drop_cache).

M07 — Filing repository

Developer. CRUD for the `filing` table, including the `item_4_02_8k` flag. **Depends on:** M03, M05. **Public interface:**

```
#[async_trait]
pub trait FilingRepo {
    async fn upsert(&self, f: &Filing) -> Result<(), RepoError>;
    async fn get(&self, accn: &AccessionNo) -> Result<Option<Filing>, RepoError>;
    async fn list_for_cik(&self, cik: &Cik) -> Result<Vec<Filing>, RepoError>;
    async fn list_unresolved_4_02(&self, cik: &Cik) -> Result<Vec<Filing>, RepoError>;
}
```

DoD: Unit tests; idempotent upsert; query plan verified to use `idx_filing_cik_filed`.

M08 — Period repository

Developer. CRUD for the `period` table; per-CIK period discovery; uniqueness handling for `fiscal_quarter = 0`. **Depends on:** M03, M05. **Public interface:**

```
#[async_trait]
pub trait PeriodRepo {
    async fn upsert_returning_id(&self, p: &Period) -> Result<i64, RepoError>;
    async fn get_id(&self, cik: &Cik, fy: i32, fq: u8) -> Result<Option<i64>, RepoError>;
    async fn list_for_cik(&self, cik: &Cik, kind: Option<PeriodKind>) -> Result<Vec<Perioc
}
```

DoD: Tests including 53-week year, `fiscal_quarter = 0` annual, multiple consecutive quarters.

M09 — Raw fact repository

Developer. CRUD for `raw_fact`; idempotent insert via natural-key UNIQUE; bulk insert API for ingestion. **Depends on:** M03, M05, M07. **Public interface:**

```
#[async_trait]
pub trait RawFactRepo {
    async fn upsert_many(&self, facts: &[RawFact]) -> Result<usize, RepoError>;
    async fn list_for_filing(&self, accn: &AccessionNo) -> Result<Vec<RawFact>, RepoError>
    async fn get(&self, id: i64) -> Result<Option<RawFact>, RepoError>;
}
```

DoD: Tests for natural-key dedup, bulk insert of 10k facts < 1s on a typical SSD.

M10 — Normalized fact repository

Developer. CRUD for `normalized_fact` + supersession-aware queries; lineage walk; `is_primary` semantics. **Depends on:** M03, M05, M08, M09. **Public interface:**

```
#[async_trait]
pub trait NormalizedFactRepo {
    /// Insert a new primary value. If a previous primary exists for
    /// (cik, metric, period_id), update its superseded_by → new id
    /// in a single transaction.
    async fn insert_primary_with_supersession(&self, n: &NormalizedFact) -> Result<i64, Re
    async fn insert_alternate(&self, n: &NormalizedFact) -> Result<i64, RepoError>;
```

```
    async fn current_value(&self, cik: &Cik, metric: Metric, period_id: i64) -> Result<Opt
    async fn current_series(&self, cik: &Cik, metric: Metric, kind: PeriodKind) -> Result<
    async fn supersession_chain(&self, id: i64) -> Result<Vec<NormalizedFact>, RepoError>;
}
```

DoD: Multi-step amendment test; cycle protection via trigger; partial unique index enforced.

M11 — Derived metric repository

Developer. CRUD for `derived_metric`; bulk recompute on input change. **Depends on:** M03, M05, M08. **Public interface:**

```
#[async_trait]
pub trait DerivedMetricRepo {
    async fn upsert(&self, d: &DerivedMetric) -> Result<(), RepoError>;
    async fn get(&self, cik: &Cik, formula: &str, period_id: i64) -> Result<Option<Derived
    async fn series(&self, cik: &Cik, formula: &str, kind: PeriodKind) -> Result<Vec<(Peri
}
```

DoD: Tests for `is_complete = false` rows surfacing as gaps.

M12 — Ingestion event repository

Developer. Append-only log of `ingestion_event` rows. **Depends on:** M03, M05. **Public interface:**

```
#[async_trait]
pub trait IngestionEventRepo {
    async fn record(&self, e: &IngestionEvent) -> Result<(), RepoError>;
    async fn recent(&self, cik: Option<&Cik>, limit: u32) -> Result<Vec<IngestionEvent>, R
    async fn user_visible(&self, cik: &Cik) -> Result<Vec<IngestionEvent>, RepoError>;
}
```

DoD: Read-vs-write contention test passes under WAL.

Sources (M13–M17)

M13 — SEC HTTP client

Developer. `reqwest` client with: User-Agent header, host allowlist (`www.sec.gov`, `data.sec.gov`), shared token-bucket rate limiter (5 req/s by default), exponential backoff on 429/5xx. **Depends on:** M04. **Public interface:**

```
pub struct SecClient { /* ... */ }
impl SecClient {
    pub fn new(user_agent: String, rps: u32) -> Self;
    pub async fn get_json<T: DeserializeOwned>(&self, url: &str) -> Result<T, SourceError>
    pub async fn get_bytes(&self, url: &str) -> Result<Vec<u8>, SourceError>;
}
```

DoD: Mock-server test verifying UA header, rate-limit enforcement, backoff on 429.

M14 — company_tickers fetcher

Developer. Fetch and parse `https://www.sec.gov/files/company_tickers.json`; cache locally with 7-day TTL. **Depends on:** M13. **Public interface:**

```
pub struct TickerMap { /* in-memory map */ }
impl TickerMap {
    pub async fn load(client: &SecClient, cache_dir: &Path) -> Result<Self, SourceError>;
    pub fn ticker_to_cik(&self, t: &Ticker) -> Option<Cik>;
}
```

DoD: Tests with checked-in fixture; TTL enforcement.

M15 — submissions fetcher

Developer. Fetch + parse `data.sec.gov/submissions/CIK*.json`; produce `Vec<Filing>` including 8-K Item 4.02 detection (item-list contains `"4.02"`). **Depends on:** M03, M13. **Public interface:**

```
pub async fn fetch_submissions(client: &SecClient, cik: &Cik) -> Result<Vec<Filing>, Sourc
```

DoD: Test against checked-in Apple submissions fixture; correctly flags any 8-K with `4.02` in items.

M16 — companyfacts fetcher + parser

Developer. Fetch + parse `data.sec.gov/api/xbrl/companyfacts/CIK*.json`; produce `Vec<RawFact>` with §6.2 micro-unit scaling applied. **Depends on:** M03, M13. **Public interface:**

```
pub async fn fetch_companyfacts(client: &SecClient, cik: &Cik) -> Result<Vec<RawFact>, Sou
```

DoD: Test against checked-in Apple companyfacts fixture; scaling test for USD (×1,000,000), shares (×1), USD/shares (×1,000,000); every fact carries a non-empty `accn`.

M17 — MarketDataAdapter trait + Yahoo Finance impl

Developer. Trait + Yahoo Finance default impl using `https://query1.finance.yahoo.com/v8/finance/chart/{ticker}`. **Depends on:** M03. **Public interface:**

```
#[async_trait]
pub trait MarketDataAdapter: Send + Sync {
    async fn historical_prices(&self, ticker: &Ticker, from: NaiveDate, to: NaiveDate)
        -> Result<Vec<(NaiveDate, Micro)>, SourceError>;
    async fn current_price(&self, ticker: &Ticker) -> Result<Micro, SourceError>;
}
pub struct YahooFinanceAdapter { /* ... */ }
impl MarketDataAdapter for YahooFinanceAdapter { /* ... */ }
```

DoD: Mock-server tests; price scaling correctness; graceful error on adapter unavailable.

Normalization (M18–M21)

M18 — Canonical metric catalog + concept map

Developer. Static map from `Metric` to ordered list of `(taxonomy, concept)` candidates. Populated for the §6.2 catalog. **Depends on:** M03. **Public interface:**

```
pub fn concepts_for(metric: Metric) -> &'static [(&'static str, &'static str)];
pub fn metric_for(taxonomy: &str, concept: &str) -> Option<Metric>;
```

DoD: Unit tests covering every Metric; tests asserting the §6.2 ordering for `total_debt`, `revenue`, etc.

M19 — Period reconciliation

Developer. Single-quarter derivation from year-to-date inputs, with span-aware slot classification, concept-consistency selection, and period-end-derived fiscal year. The reconciler ignores the SEC `fy` tag entirely (it carries the filing's year, not the period's; see architecture §8.2) and uses `fp` only as a position hint, classifying each duration fact into a span-aware slot (`SingleQ1..Q4` ≤110 days, `YtdH1` 150–210 days, `Ytd9M` 240–290 days, `Fy` ≥340 days). Within each `(metric, fiscal_year)` it picks one source XBRL concept (highest slot coverage; catalog-priority tie-break) so derivations like `Q4 = FY − 9M` cannot mix concept scopes (e.g. `DepreciationAndAmortization` annual-only vs `DepreciationAmortizationAndAccretionNet` quarterly-with-accretion). 53-week detection is on `end_date − start_date > 364 days`. **Depends on:** M03, M08. **Public interface:**

```
pub fn reconcile_quarters(raw: &[RawFact], fye_mmdd: &str)
    -> (Vec<QuarterValue>, Vec<RawFact>);

pub struct QuarterValue {
    pub metric: Metric,
    pub cik: Cik,
    pub fy: i32,
    pub fq: u8,
    pub period_start: NaiveDate,
    pub period_end: NaiveDate,
    pub source_fact_id: i64,
    pub value: i64,
    pub source_kind: SourceKind,
    pub derived: bool,
}

// Period helpers used by the orchestrator for instant facts:
impl Period {
    pub fn compute_fiscal_year(end: NaiveDate, fye_mmdd: &str) -> i32;
    pub fn compute_fiscal_quarter(end: NaiveDate, fye_mmdd: &str) -> Option<u8>;
}
```

DoD: Tests for: pure annual, pure quarterly, YTD-only Q3 → derive Q3, single-quarter Q2 wins over derived when both filed under the same `fp`, concept-with-more-coverage wins when two XBRL concepts share a metric, periods-segregated-by-period-end-year-not-SEC-`fy`-tag, 53-week year, FYE change mid-history.

M20 — Unit + sign normalization

Developer. Pure functions converting raw facts to canonical metric values applying §6.2 sign conventions. **Depends on:** M03, M18. **Public interface:**

```
pub fn canonical_value(metric: Metric, raw: &RawFact) -> Result<Micro, PipelineError>;
```

DoD: Tests for CapEx sign-flip, EPS sign passthrough, currency mismatch error.

M21 — Resolution rules

Developer. Selects the primary `RawFact` for `(cik, metric, period_id)` per §8.1 ordering: amendment > original > catalog primary > xbrl_api > xbrl_xml. **Depends on:** M03, M07, M18. **Public interface:**

```
pub fn resolve(metric: Metric, candidates: &[RawFact], filings: &HashMap<AccessionNo, Fil
    -> Option<(usize /* primary index */, Vec<usize> /* alternates */)>;
```

DoD: Tests for: original-only, original + amendment (amendment wins), multiple concepts (catalog primary wins), api+xml duplicate (api wins).

Pipeline (M22–M27)

M22 — Pipeline orchestrator

Developer. Coordinates Discover → Download → Parse → Normalize → Persist; emits progress events; rate-limited concurrency. **Depends on:** M23–M27, M30. **Public interface:**

```
pub struct IngestionJob { pub id: String, pub cik: Cik, /* ... */ }
pub async fn ingest_company(
    deps: &PipelineDeps,
    ticker: &Ticker,
    progress: ProgressSink,
) -> Result<IngestionSummary, PipelineError>;
```

DoD: End-to-end test using checked-in fixtures: ticker → all stages → DB rows present.

M23 — Discover stage

Developer. Ticker → CIK → submissions list. Pure async function. **Depends on:** M14, M15. **Public interface:** `pub async fn discover(client, ticker) -> Result<DiscoverOutput, ...>;` **DoD:** Unit test with fixture.

M24 — Download stage

Developer. CIK → companyfacts JSON path on disk + filings index. **Depends on:** M16. **Public interface:** `pub async fn download(client, cik, cache_dir) -> Result<DownloadOutput, ...>;` **DoD:** Idempotent (re-running is a no-op when cached).

M25 — Parse stage

Developer. Companyfacts JSON → `Vec<RawFact>`. **Depends on:** M16. **Public interface:** `pub fn parse(input: DownloadOutput) -> Result<ParseOutput, ...>;` **DoD:** Output count matches a known fixture's expected count.

M26 — Normalize stage

Developer. RawFacts + ConceptMap → NormalizedFacts + Diagnostics. **Depends on:** M18, M19, M20, M21. **Public interface:** `pub fn normalize(input: ParseOutput) -> Result<NormalizeOutput, ...>;` **DoD:** Test that asserts no metric in the catalog produces zero values for Apple's history; sign conventions hold.

M27 — Persist stage

Developer. Atomic write: filings, periods, raw_facts, normalized_facts (with supersession), diagnostics. **Depends on:** M06, M07, M08, M09, M10, M12. **Public interface:** `pub async fn persist(deps, normalized) -> Result<PersistOutput, ...>;` **DoD:** Crash-recovery test; idempotent re-persist of same accession.

Derived metrics (M28)

M28 — Derived metric registry + formulas

Developer. Two persistence styles:

- **Persisted at ingest, written to `derived_metric`:** `historical_market_cap_v1`, `bank_revenue_v1` (steps 3–4 of architecture §8.1, run only when no direct `Revenue` exists for the period; positivity-guarded).
- **Read-time only, computed from `normalized_fact` rows:** `total_debt_v1`, `gross_profit_v1`, `capital_expenditures_v1` (the PP&E-roll-forward fallback `ΔPP&E_net + D&A`, used when no explicit cash-flow CapEx is tagged), `free_cash_flow_v1` (`net_income + depreciation_amortization − capital_expenditures`; all three inputs required), and `operating_margin_v1` (`operating_income ÷ revenue`, stored as a ratio ×1e6; omitted when revenue ≤ 0). Read-time avoids stale-cache hazards when an input is superseded. The pure formulas live in `derived` (unit-tested); the per-period series assembly lives in `derived::series`, parameterized over the repository traits so IPC and integration tests share one path.
- **Live-only, never persisted:** `current_market_cap_v1`.

> **Note (revision).** An earlier draft listed `fcf_v1` as *persisted at ingest*. Free cash flow moved to the **read-time** family alongside `total_debt` / `gross_profit` / `capital_expenditures` for the same reason: a persisted sum goes stale when any of net income, depreciation & amortization, or capital expenditures is superseded by a later filing. `operating_margin_v1` was added in the same revision (PRD FR-033).

Depends on: M03, M11, M17. **Public interface:**

```
pub trait Formula: Send + Sync {
    fn id(&self) -> &'static str;
```

```
    fn inputs(&self) -> &[Metric];
    fn compute(&self, inputs: &MetricInputs, ctx: &FormulaCtx) -> DerivedResult;
}
pub fn registry() -> Vec<Box<dyn Formula>>;

// Read-time merge in IPC: revenue queries union direct primary
// revenue with bank_revenue_v1 derived rows.
pub async fn revenue_aware_series(/* ... */) -> Result<MetricSeries, IpcError>;
```

DoD: Per-formula unit tests with synthesized inputs; `is_complete = false` returned cleanly when any input missing. Bank-revenue chain: step 3 picked when both inputs present; step 4 picked when `NetInterestIncome` absent but the `(II0, IE, NoniI)` triple is present; non-positive derived value skipped + warned. Free cash flow and operating margin: pure-formula unit tests in `derived` (exact micro-unit values, overflow saturation, undefined-margin handling) plus a production-mode end-to-end accuracy test (`integration_derived_metrics`) that runs the read-time series over the real ingested DB and asserts (a) internal consistency — every emitted value re-derives from independently-fetched component series — (b) coverage matches exactly the periods with all inputs, and (c) hand-verified figures (Zoetis FY2025, Dollar General FY2026, lululemon FY2026).

IPC (M29–M30)

M29 — Tauri command catalog

Architect + Developer. All `#[tauri::command]` entry points, fully typed (request + response). Each command returns `Result<T, AppError>` . **Depends on:** M03, M04, M22, all repos. **Public interface:**

```
// Commands (request → response):
add_company(ticker: String) -> Company
remove_company(cik: String, drop_cache: bool) -> ()
list_companies() -> Vec<Company>
refresh_company(cik: String) -> IngestionSummary
get_dashboard(cik: String) -> DashboardPayload
get_metric_history(cik: String, metric: String, kind: String) -> Vec<(Period, Micro)>
get_lineage(normalized_fact_id: i64) -> LineageRecord
get_ingestion_events(cik: Option<String>, limit: u32) -> Vec<IngestionEvent>
```

DoD: TypeScript-typed bindings (`src/api/types.ts`) generated/maintained alongside the Rust definitions; round-trip tests for each command.

M30 — Event channel

Developer. Tauri events for ingestion progress (`ingestion://progress/{job}`), errors, refresh completion. **Depends on:** M01, M22. **Public interface:** typed event payloads documented in `src/api/events.ts`. **DoD:** UI subscribes; payloads serialize correctly.

Frontend foundation (M31–M34)

M31 — React/Vite/TS setup

Developer. Vite + React 18 + TS + path aliases. **Depends on:** M01. **DoD:** `pnpm build` produces a static bundle Tauri can serve.

M32 — Typed IPC client

Developer. TS wrapper around Tauri's `invoke()` with one typed function per command (M29). **Depends on:** M29, M31. **Public interface:** `src/api/client.ts` exporting one async function per command. **DoD:** Each function's TS type matches the Rust command signature; mock-IPC tests.

M33 — TanStack Query hooks

Developer. `useCompanies()`, `useDashboard(cik)`, `useMetricHistory(cik, metric, kind)`, `useLineage(id)`, etc. **Depends on:** M32. **DoD:** Hooks compile; basic render tests.

M34 — Tailwind + theme

Developer. Tailwind config; tokens (font, color, spacing); dark mode default per architecture §11.4. **Depends on:** M31. **DoD:** Dev server renders styled components.

UI features (M35–M42)

M35 — Home: SavedCompaniesList + AddTickerDialog

Developer. Route `/`. List, add, remove. **Depends on:** M33. **DoD:** Add ticker → row appears; remove → row disappears.

M36 — Dashboard layout

Developer. Route `/c/:ticker`. Three-row layout: SummaryWidgets / ChartGrid / StatementsTable; lineage drawer slot. **Depends on:** M33. **DoD:** Renders for an ingested company; renders skeletons during load.

M37 — SummaryWidgets

Developer. Revenue, Net income, Cash, Total debt, FCF, Historical market cap; sparkline per widget. **Depends on:** M33, M36. **DoD:** Renders against fixture data; numbers display in dollars (converted from

micro-units at presentation).

M38 — ChartGrid

Developer. ECharts time-series with annual/quarterly toggle, 10y/20y range. **Depends on:** M33, M36. **DoD:** Renders chart with annual + quarterly modes; gaps shown explicitly.

M39 — StatementsTable

Developer. TanStack Table virtualized; rows = line items; columns = periods. Drill-down opens lineage drawer. **Depends on:** M33, M36, M40. **DoD:** Dense table with proper number formatting; row click → lineage open.

M40 — LineagePanel

Developer. Side drawer showing filing accession, form, date, XBRL concept, original value, FX conversion, supersession chain. **Depends on:** M33. **DoD:** Opens for a given normalized_fact_id; renders all lineage fields.

M41 — DiagnosticsTab

Developer. Renders `ingestion_event` rows filtered by level/stage. (Deferred — interface only; minimal placeholder UI.) **Depends on:** M33. **DoD (for slice):** route exists, renders an empty state.

M42 — Loading + error states

Developer. Skeleton placeholders, explicit "missing data" markers, IPC error boundary. **Depends on:** M33. **DoD:** Storybook-style render tests; error state rendered for an unknown ticker.

Tests (M43–M45)

M43 — Unit test scaffolding

Tester. `cargo test` for Rust (per-module); `vitest` for the React side. Coverage configuration. **Depends on:** M01. **DoD:** Both runners green on a placeholder test; CI command documented.

M44 — Integration test fixtures

Tester. Checked-in `companyfacts.json` for AAPL pinned to a specific date; checked-in `submissions.json`; one mocked Yahoo Finance response. **Depends on:** M01. **DoD:** Fixtures present at `tests/fixtures/`; date pinned in a `FIXTURES.md`; integration test using them passes.

M45 — E2E test plan + harness

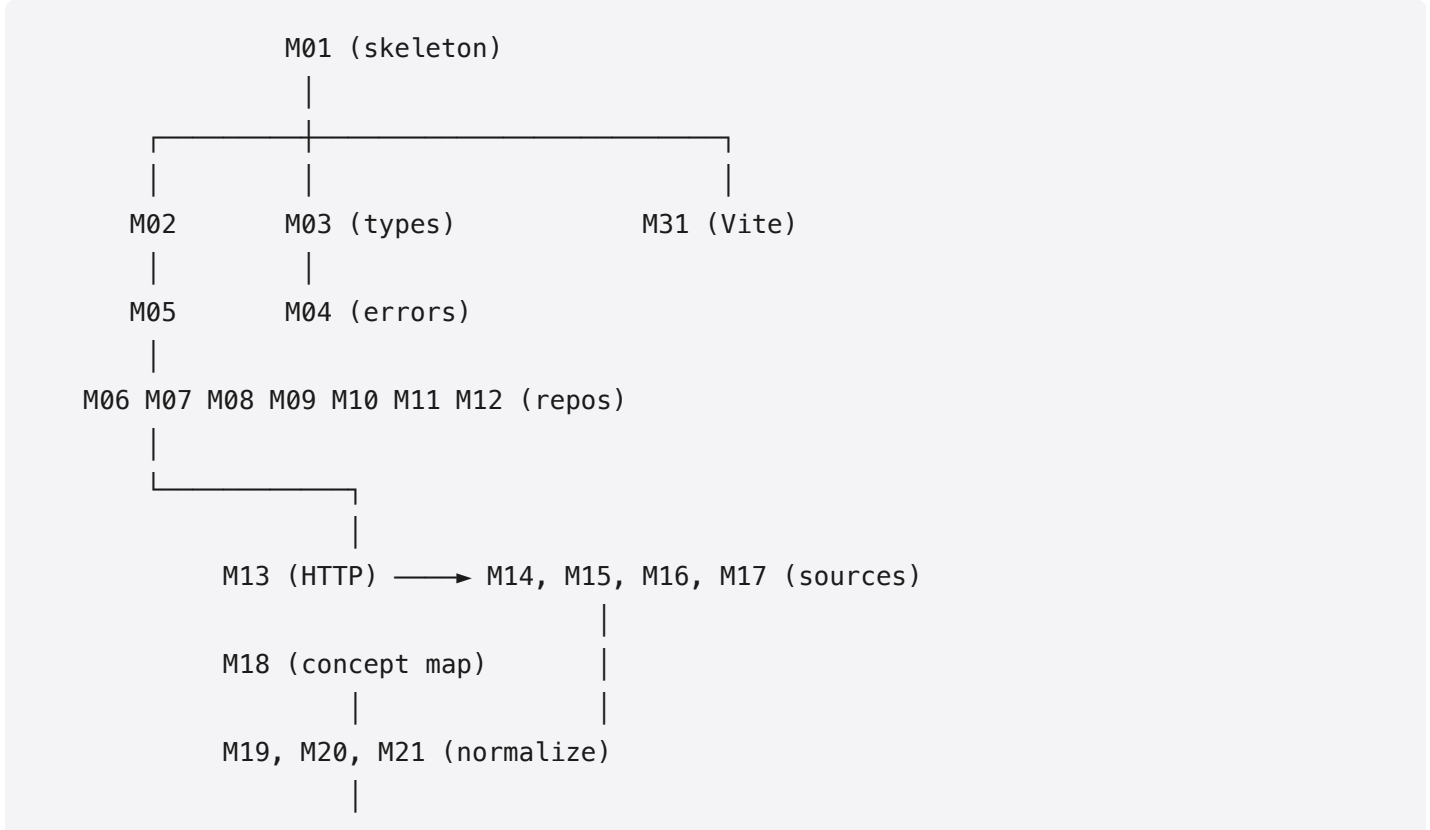
Tester. End-to-end tests **from the perspective of a human user**, using Tauri's `tauri-driver` (or a mock harness running the UI against a fake IPC).

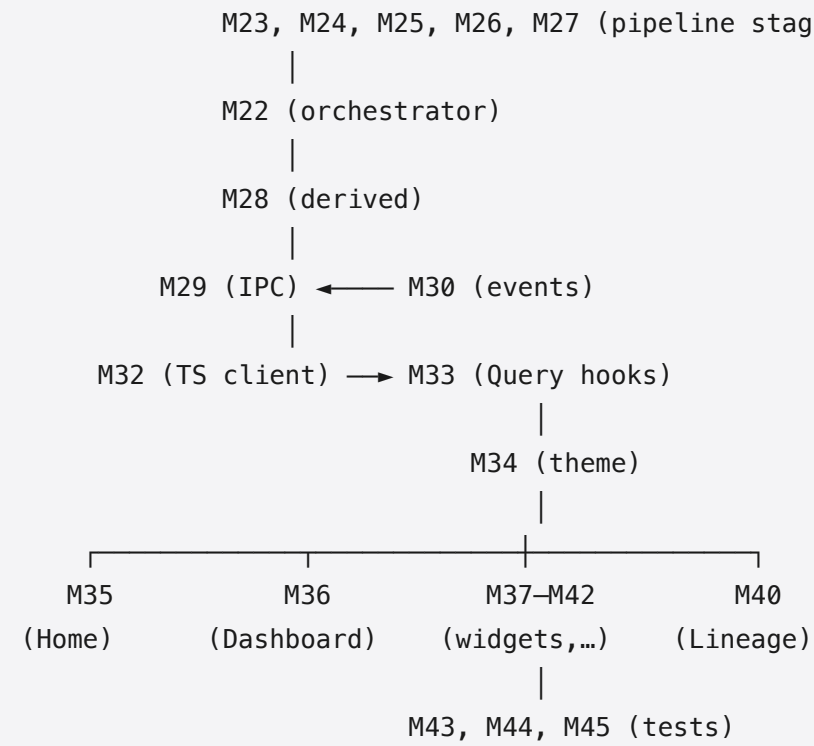
E2E plan covers:

1. **First-run flow:** App launches → empty home screen → user types `AAPL` → ticker validates → ingestion runs (using fixture, not live SEC) → dashboard appears.
2. **Dashboard navigation:** User views revenue widget → clicks chart → toggles annual/quarterly → drills into a metric → lineage drawer shows the source filing.
3. **Refresh flow:** User clicks refresh → progress events surface → dashboard updates if there is new data.
4. **Offline flow:** Disconnect (mock) → previously ingested companies still navigable; refresh button disabled with explanatory message.
5. **Error flow:** Add an unknown ticker → user-visible error message with details.
6. **Lineage correctness:** For Apple FY2023 revenue, the lineage drawer shows accession `0000320193-23-XXXXXX`, form `10-K`, the XBRL concept that produced the value, and the value preserved exactly.
7. **Restatement (deferred but specced):** Ingest a fixture with a 10-K/A → the chart for the affected period shows the restated value; lineage drawer walks the supersession chain.

Depends on: M01, M44, the slice modules. **DoD:** Each scenario above has a runnable test (or, where the harness can't yet drive the UI, a documented manual test script). Scenarios 1–5 must pass automatically.

6. Dependency graph





7. Build order (waves of parallel work)

- **Wave 0 (sequential):** M01 (architect-led skeleton).
- **Wave 1 (parallel):** M02, M03, M04, M31.
- **Wave 2 (parallel):** M05, M13, M14, M18, M32, M34.
- **Wave 3 (parallel):** M06–M12 (repos), M15, M16, M17, M19, M20, M21, M33.
- **Wave 4 (parallel):** M23, M24, M25, M26, M27, M28, M29, M30, M35–M42.
- **Wave 5 (sequential):** M22 (orchestrator wires the stages).
- **Wave 6 (parallel):** M43, M44, M45 (tests at every level).

A team of agents can max-parallel each wave; only Wave 0 and Wave 5 are sequential.

8. Definition of done — product level

The product is "done for V1" when, on a Mac with Rust + Node installed:

1. `pnpm install && pnpm tauri dev` launches the app.
2. The user adds `AAPL`; ingestion runs against live SEC EDGAR; the dashboard shows revenue, net income, cash, total debt, FCF, historical market cap.
3. Annual / quarterly toggle works; chart range default 10y; periods with no data render as gaps.
4. Drilling into a metric opens the lineage drawer with filing accession, form, date, and XBRL concept.
5. `cargo test` passes; `pnpm test` passes.
6. The E2E test harness runs scenarios 1–5 from §M45 to green.

7. Offline launch (no network) keeps the dashboard navigable for previously ingested companies.

9. Out-of-V1-implementation-slice notes

The following modules are specified above but will be stub or deferred for the first implementation pass; the tech spec stays the source of truth so a follow-up implementation can complete them without redesign:

- M16 fallback to raw XBRL XML (the `xbrl_xml` `source_kind` path).
- 8-K Item 4.02 HTML parser (interface stubbed; no real extraction).
- Bundled ECB FX dataset (FX rate columns present in schema; conversion stubbed for non-USD filers).
- Amendment-coverage-gap UI surfacing (logic in M27, UI deferred).
- Restatement-resolution join-table population at amendment ingestion (interface stubbed).
- Full Diagnostics tab (M41 minimal placeholder).

These items are tracked in `docs/followup.md` (created at the end of the implementation pass).